Band pass filter

Write a function **BandPassFilter**, which takes as input a [1 x N] vector **x** (our signal), a scalar **Fs** (the sampling frequency of the signal), a high cut-off frequencies for the passband of the filter), and an integer W (the window size of the filter). The function should use the 'fir1' fu using the 'conv' function to create the output filtered signal **y**. Use the 'same' argument with the 'conv' function so that **y** and **x** have the sa

The steps are:

- Create the filter using 'fir1'
- Apply the filter using 'conv'

Note: The larger the window size **W**, the better the filter performance. For audio frequencies with high sampling rates, **W** may need to be performance. We will see how to visualize filter characteristics to verify filter performance in a following lesson.

Function

Save Reset MATLAB Documentation (https://www.mathworks.com/help/)

```
function y = BandPassFilter(x, Fs, freq_range, W)
    %Create the filter using 'fir1'
    freq_range = freq_range / (Fs / 2);
    filt = fir1(W, freq_range);
    %Apply the filter using 'conv'
    y = conv(x, filt, 'same');
end
```

Code to call your function

Reset

```
Fs=10;
x1 = repmat([1 0 -1 0],[1,10]); % this is a 2.5 Hz signal
x2 = repmat([1 -1],[1,20]); % this is a 5 Hz signal
x = x1 + x2;
freq_range = [2,3]; % filter out the 5 Hz but keep the 2.5 Hz component
W = 10;
%%%%%%%%%%%%%%%%%
y = BandPassFilter(x, Fs, freq_range, W);
%%%%%%%%%%%%%%%%%
% plot x1, x2, x=x1+x2, and BandPassFilter output
t = 0:1/Fs:(length(y)-1)/Fs;
plot(t(1:Fs),x1(1:Fs),'bo-')
hold on
plot(t(1:Fs),x2(1:Fs),'r*-')
plot(t(1:Fs),x(1:Fs),'g')
plot(t(1:Fs),y(1:Fs),'m--', 'LineWidth', 2)
legend('x1-2.5Hz','x2-5.0Hz','x=x1+x2','output')
```

▶ Run Function

Assessment: All Tests Passed

Submit

✔ **Is output correct with random valued signal?**